

L2 - 4.3. Detecting AD Credential Attacks

Task 1 - Introduction

This room covers five techniques that bridge the gap between "attacker has a foothold" and "attacker owns the domain."

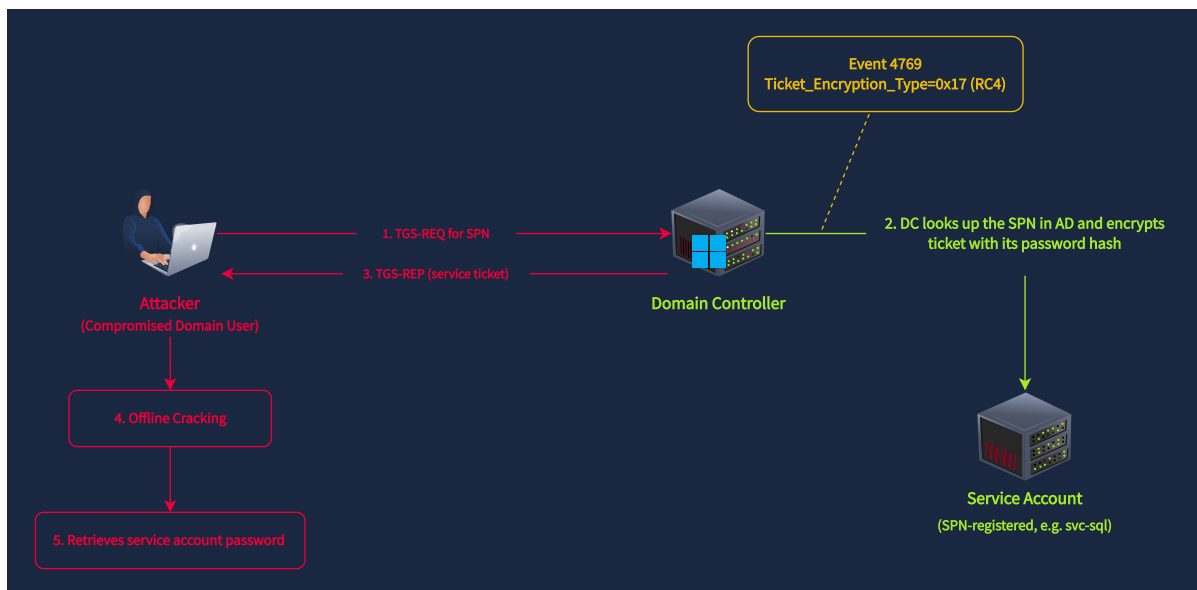
- Kerberoasting and AS-REP Roasting abuse Kerberos to crack passwords offline.
- LSASS dumping extracts credentials directly from memory.
- DCSync impersonates a domain controller to pull every password hash in the directory.
- NTDS.dit extraction copies the AD database file directly from the domain controller's disk.

Task 2 - Detecting Kerberoasting

Kerberos is a network authentication protocol that allows individuals and systems to securely prove their identity to one another over a non-secure network using encrypted **tickets**. It serves as the default authentication protocol for Microsoft Active Directory domains

Kerberoasting is a post-exploitation credential theft attack in Active Directory (AD) environments where an attacker requests Kerberos tickets for service accounts and cracks their passwords offline. Attackers target service accounts through Kerberoasting because these accounts often have weak passwords and elevated privileges.

How Do Accounts End Up Kerberoastable? Accounts become Kerberoastable when administrators configure service identities such as SQL Server or web applications, using user accounts with weak, memorable passwords and register Service Principal Names (SPNs) on them. Because Active Directory Domain Controllers automatically issue Service Tickets (TGS) encrypted with the service account's password hash to *any* requesting domain user without checking access intent, an attacker can simply request these tickets and crack weak passwords offline to recover plain-text credentials. The diagram below illustrates this flow:



TGS-REQ (Ticket Granting Service Request) is a Kerberos protocol message sent by a client to the Ticket Granting Server (TGS)—a key role inside the Domain Controller

AES-256 (Advanced Encryption Standard with a 256-bit key) is a symmetric-key block cipher widely considered the global standard for securing sensitive data at rest and in transit.

RC4 (Rivest Cipher 4, also known as ARC4) is a legacy, symmetric **stream cipher**.

A **Service Principal Name (SPN)** is a **unique identifier** used by the **Kerberos authentication protocol** to associate a specific service instance ((like SQL Server, IIS, or SMB) with a **service logon account** in **Active Directory**.

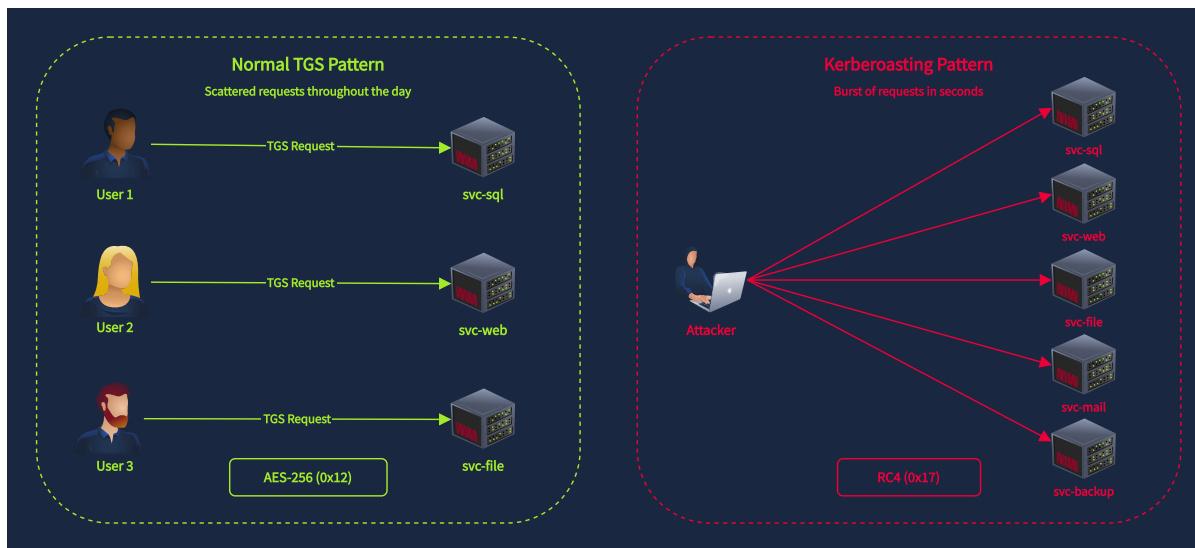
The detection centers on Event 4769 (Kerberos Service Ticket Requested), which the DC logs when it processes a TGS-REQ. Modern AD environments default to AES-256 encryption (`0x12`) for Kerberos tickets. Most Kerberoasting tools, including Rubeus and Impacket's `GetUserSPNs.py` , downgrade the request to RC4 encryption (`0x17`) because RC4 hashes are much faster to crack. A TGS request with `Ticket_Encryption_Type=0x17` in an environment that uses AES is a strong indicator. Here are the fields in Event 4769 (A Kerberos service ticket was requested) that matter for detection:

Field	What It Contains	Why It Matters
Service_Name	The SPN (service Principal Name) being requested (e.g., svc-sql)	Identifies which service account is being targeted
Ticket_Encryption_Type	0x12 (AES) or 0x17 (RC4)	RC4 requests are the primary Kerberoasting signal
Account_Name	The account requesting the ticket	Identifies the compromised user running the attack

Field	What It Contains	Why It Matters
Client_Address	Source IP of the request	Identifies the attacker's machine

Important note: The `Client_Address` field in Event 4769 often uses IPv6-mapped IPv4 notation on modern Windows domain controllers. Instead of displaying `10.5.90.1`, the field shows `::ffff:10.5.90.1`. This is standard behavior, not an error. The actual IPv4 address is the portion after `::ffff:`, so `::ffff:10.5.90.1` means the source IP is `10.5.90.1`.

Walkthrough: Normal Kerberos traffic includes a lot of Event 4769 entries. Computer accounts (names ending in `$`) and `krbtgt` requests generate high volumes as part of regular domain operations. We exclude both to focus on service accounts that could be Kerberoasting targets. Our primary filter is `Ticket_Encryption_Type=0x17` since that's the RC4 downgrade signal. The diagram below contrasts normal TGS patterns with a Kerberoasting attack:



```
index=task2 EventCode=4769 Ticket_Encryption_Type=0x17 Service_Name!
="*$" Service_Name!="krbtgt"
| table _time, Account_Name, Service_Name, Ticket_Encryption_Type, Cl
ient_Address
| sort _time
```

_time	Account_Name	Service_Name	Ticket_Encryption_Type	Client_Address
2026-03-14 19:12:43	sarah.chen@TRYHATMESTUDIOS.THM	svc-print	0x17	::ffff:10.10.45.12
2026-03-14 19:12:43	sarah.chen@TRYHATMESTUDIOS.THM	svc-mail	0x17	::ffff:10.10.45.12
2026-03-14 19:12:43	sarah.chen@TRYHATMESTUDIOS.THM	svc-backup	0x17	::ffff:10.10.45.12
2026-03-14 19:12:43	sarah.chen@TRYHATMESTUDIOS.THM	svc-share	0x17	::ffff:10.10.45.12
2026-03-14 19:12:43	sarah.chen@TRYHATMESTUDIOS.THM	svc-sql	0x17	::ffff:10.10.45.12
2026-03-14 19:12:43	sarah.chen@TRYHATMESTUDIOS.THM	svc-web	0x17	::ffff:10.10.45.12

Multiple TGS requests with RC4 encryption, all from the same account, targeting different service accounts, within a short time window. That pattern is textbook Kerberoasting.

```
index=task2 EventCode=4769 Ticket_Encryption_Type=0x17 Service_Name!
="*$" Service_Name!="krbtgt"
| stats dc(Service_Name) as targeted_services count by Account_Name,
Client_Address
```

Account_Name	Client_Address	targeted_services	count
sarah.chen@TRYHATMESTUDIOS.THM	::ffff:10.10.45.12	■	■

Detection evasion: RC4-based detection alone isn't enough for the long term. Volume-based detection catches both RC4 and AES Kerberoasting because tools like `GetUserSPNs.py` and Rubeus request tickets for every SPN-holding account in the domain by default, producing a burst of TGS requests that stands out against normal activity regardless of the encryption type used. The following query groups TGS requests into 5-minute windows and flags any account that requested tickets for more than 5 unique service accounts within a window:

```
index=task2 EventCode=4769 Service_Name!="*$" Service_Name!="krbtgt"
| bin _time span=5m
| stats dc(Service_Name) as unique_spns count by Account_Name, Client
_Address, _time
| where unique_spns > 5
```

Account_Name	Client_Address	_time	unique_spns	count
sarah.chen@TRYHATMESTUDIOS.THM	::ffff:10.10.45.12	2026-03-14 19:10:00	■	■

The `!` excludes computer accounts (which end with a `$`) and the `krbtgt` account, as neither is targeted in Kerberoasting.

Answer the questions below

How many service accounts were targeted by Kerberoasting?

```
index=task2 EventCode=4769 Ticket_Encryption_Type=0x17 Service_Name!
="*$" Service_Name!="krbtgt"
| table _time, Account_Name, Service_Name, Ticket_Encryption_Type, Cl
ient_Address
| sort _time
```

What account requested the service tickets? (Answer Format: username only, without @domain)

emma.wilson

What source IP initiated the Kerberoasting?

10.5.90.1

New Search

Save As ▾

Create Table View

Close

1 index=task2 EventCode=4769 Ticket_Encryption_Type=0x17 Service_Name!="*\$" Service_Name!="krbtgt"

All time ▾ 🔍

2 | table _time, Account_Name, Service_Name, Ticket_Encryption_Type, Client_Address

3 | sort _time

✓ 9 events (before 8/18/26 7:46:56.000 AM) No Event Sampling ▾

Job ▾

|||

↗

📄

⬇️

Smart Mode ▾

Events

Patterns

Statistics (9)

Visualization

Show: 20 Per Page ▾

Format ▾

☒ Preview: On

_time ↕	Account_Name ↕	Service_Name ↕	Ticket_Encryption_Type ↕	Client_Address ↕
2026-03-14 19:12:43	emma.wilson@TRYHATSTUDIOS.THM	svc-print	0x17	::ffff:10.5.90.1
2026-03-14 19:12:43	emma.wilson@TRYHATSTUDIOS.THM	svc-mail	0x17	::ffff:10.5.90.1
2026-03-14 19:12:43	emma.wilson@TRYHATSTUDIOS.THM	svc-backup	0x17	::ffff:10.5.90.1
2026-03-14 19:12:43	emma.wilson@TRYHATSTUDIOS.THM	svc-share	0x17	::ffff:10.5.90.1
2026-03-14 19:12:43	emma.wilson@TRYHATSTUDIOS.THM	svc-sql	0x17	::ffff:10.5.90.1
2026-03-14 19:12:43	emma.wilson@TRYHATSTUDIOS.THM	svc-web	0x17	::ffff:10.5.90.1
2026-03-14 19:12:44	emma.wilson@TRYHATSTUDIOS.THM	svc-report	0x17	::ffff:10.5.90.1
2026-03-14 19:12:44	emma.wilson@TRYHATSTUDIOS.THM	svc-app	0x17	::ffff:10.5.90.1
2026-03-14 19:12:44	emma.wilson@TRYHATSTUDIOS.THM	svc-ftp	0x17	::ffff:10.5.90.1

Task 3 - Detecting AS-REP Roasting

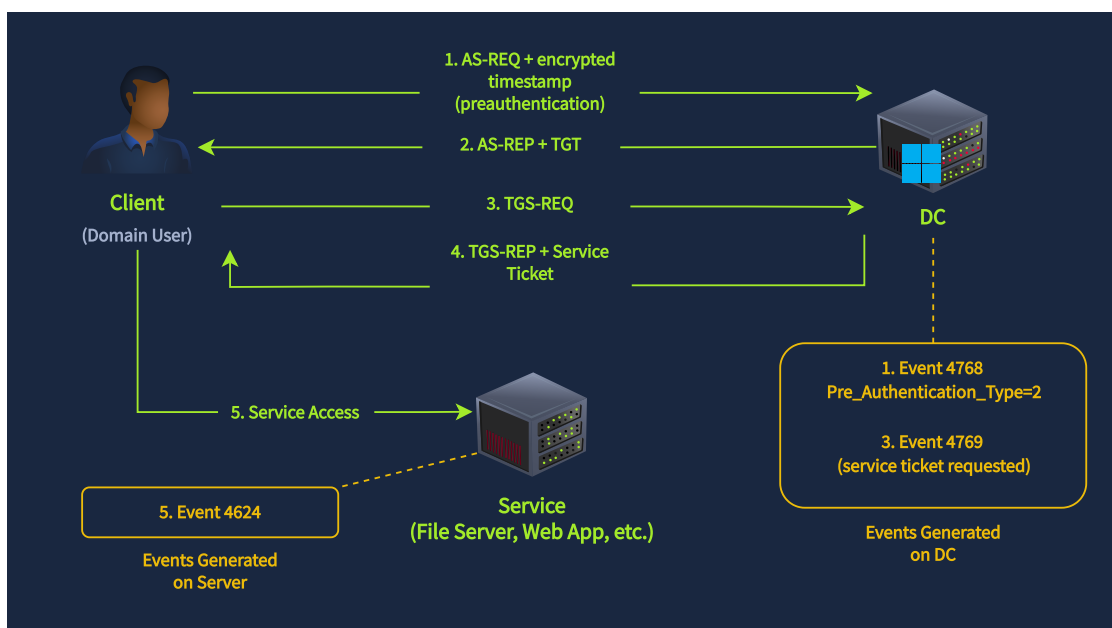
AS-REQ (Authentication Server Request) is the initial **Kerberos** authentication message sent by a client to the **Active Directory** Domain Controller (specifically the Key Distribution Center) to request a **Ticket Granting Ticket (TGT)**. This request is transmitted in plaintext and contains the client's principal, service principal, IP list, and requested lifetime, serving as the first step in the Kerberos authentication flow before pre-authentication data is validated.

AS-REP Roasting is a credential access attack against **Active Directory** that targets user or service accounts configured with **Kerberos pre-authentication disabled**. By exploiting this misconfiguration, attackers can request an **Authentication Service Response (AS-**

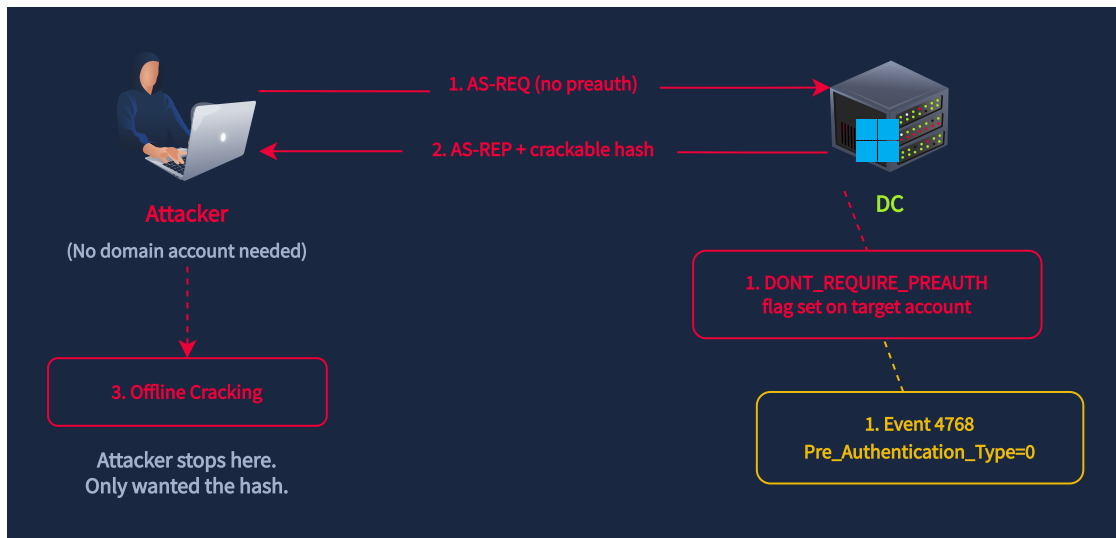
REP) from the Key Distribution Center (KDC) without providing valid credentials, receiving an encrypted message containing a Ticket Granting Ticket (TGT).

Kerberoasting targets service accounts that have SPNs registered. AS-REP Roasting targets a different weakness entirely: user accounts with preauthentication disabled. It doesn't require SPNs, and unlike Kerberoasting, the attacker doesn't even need valid domain credentials to perform it. The underlying flaw is the same, though: the DC hands back encrypted material that the attacker can crack offline to recover the plaintext password. The difference is what triggers the DC to hand it over and what it's encrypted with.

Under normal Kerberos authentication, when a user requests a TGT (ticket-granting ticket), the domain controller requires them to prove they know their password first. The user encrypts a timestamp with their password hash and sends it along with the AS-REQ. The DC decrypts it, verifies the timestamp is recent, and only then issues the TGT. The DC logs Event 4768 (Kerberos TGT Request) with `Pre_Authentication_Type=2` (represents `PA-ENC-TIMESTAMP` (Encrypted Timestamp Pre-Authentication)), followed by Event 4769 when the user requests a service ticket, and Event 4624 when they log onto the target service. The diagram below shows this normal authentication flow:

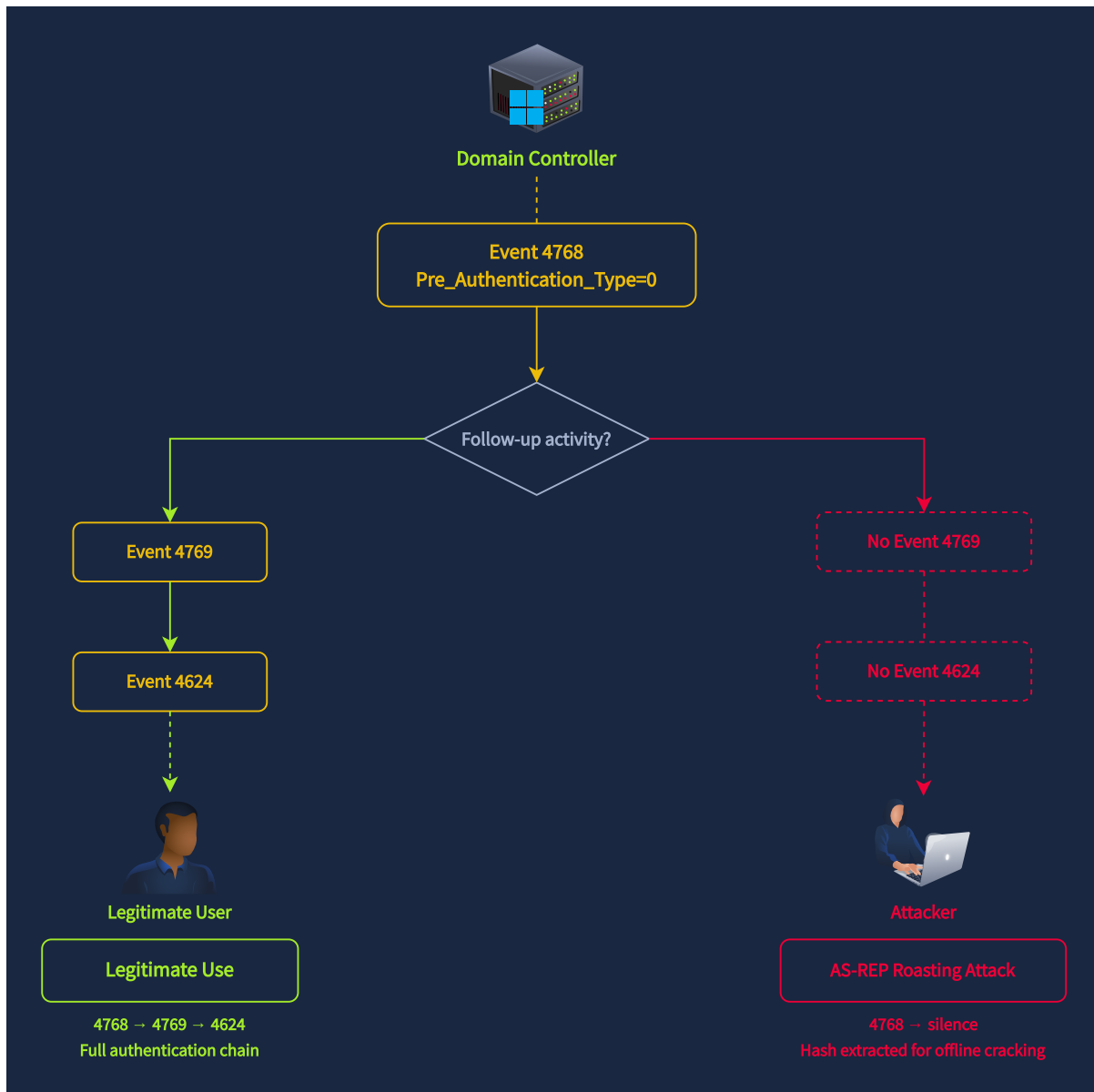


On the other hand, when an account has the `DONT_REQUIRE_PREAUTH` flag enabled, the DC skips that verification step. It issues the TGT without first confirming the requester knows the password. The AS-REP that comes back is encrypted with the account's password hash, which means an attacker can take it offline and crack it to recover the plaintext password. An attacker doesn't even need a valid domain account. They only need to know the username of an account with preauthentication disabled. The diagram below shows how this attack bypasses the preauthentication step:



This misconfiguration exists because some legacy applications, like older ERP or payroll systems, fail Kerberos authentication unless preauth is disabled for their service account. Admins might disable it to get the application working and then forget about it.

The key difference between AS-REP Roasting and the normal flow is that the attacker requests the TGT purely to extract the crackable hash. They never request a service ticket, and they never log onto anything. So the DC logs Event 4768 with `Pre_Authentication_Type=0`, (indicates no pre-authentication was used) but there is no 4769 and no 4624. The diagram below highlights this difference:



Walkthrough: Let's start by looking at all TGT request events to see what normal traffic looks like:

```
index=task3 EventCode=4768
| table _time, Account_Name, Pre_Authentication_Type, Ticket_Encryption_Type, Client_Address
| sort _time
```

_time	Account_Name	Pre_Authentication_Type	Ticket_Encryption_Type	Client_Address
2026-03-15 02:02:20	liam.patel	2	0x12	::ffff:10.5.50.20
2026-03-15 02:02:20	jessica.martinez	2	0x12	::ffff:10.5.50.15
2026-03-15 02:02:20	luke.sullivan	2	0x12	::ffff:10.5.50.10

Most entries show `Pre_Authentication_Type=2` (encrypted timestamp) with AES-256 encryption (`0x12`), which is normal Kerberos authentication. You may also see `Pre_Authentication_Type=15` (indicates that the user authenticated using a **Smart Card, X.509 Certificate**) for certificate-based authentication in your organization's traffic. The entry with `Pre_Authentication_Type=0` (indicates **no pre-authentication was used**) and RC4 encryption (`0x17`) is the anomaly we're investigating. Let's isolate it:

```
index=task3 EventCode=4768 Pre_Authentication_Type=0
| table _time, Account_Name, Pre_Authentication_Type, Ticket_Encryption_Type, Client_Address
```

_time	Account_Name	Pre_Authentication_Type	Ticket_Encryption_Type	Client_Address
2026-03-15 02:06:42	james.murphy	0	0x17	::ffff:10.10.45.12
2026-03-15 02:06:42	james.murphy	0	0x17	::ffff:10.10.45.12

This returns the AS-REP Roasting attempt. The `Pre_Authentication_Type=0` entry tells us this account has preauthentication disabled, and someone requested a TGT for it.

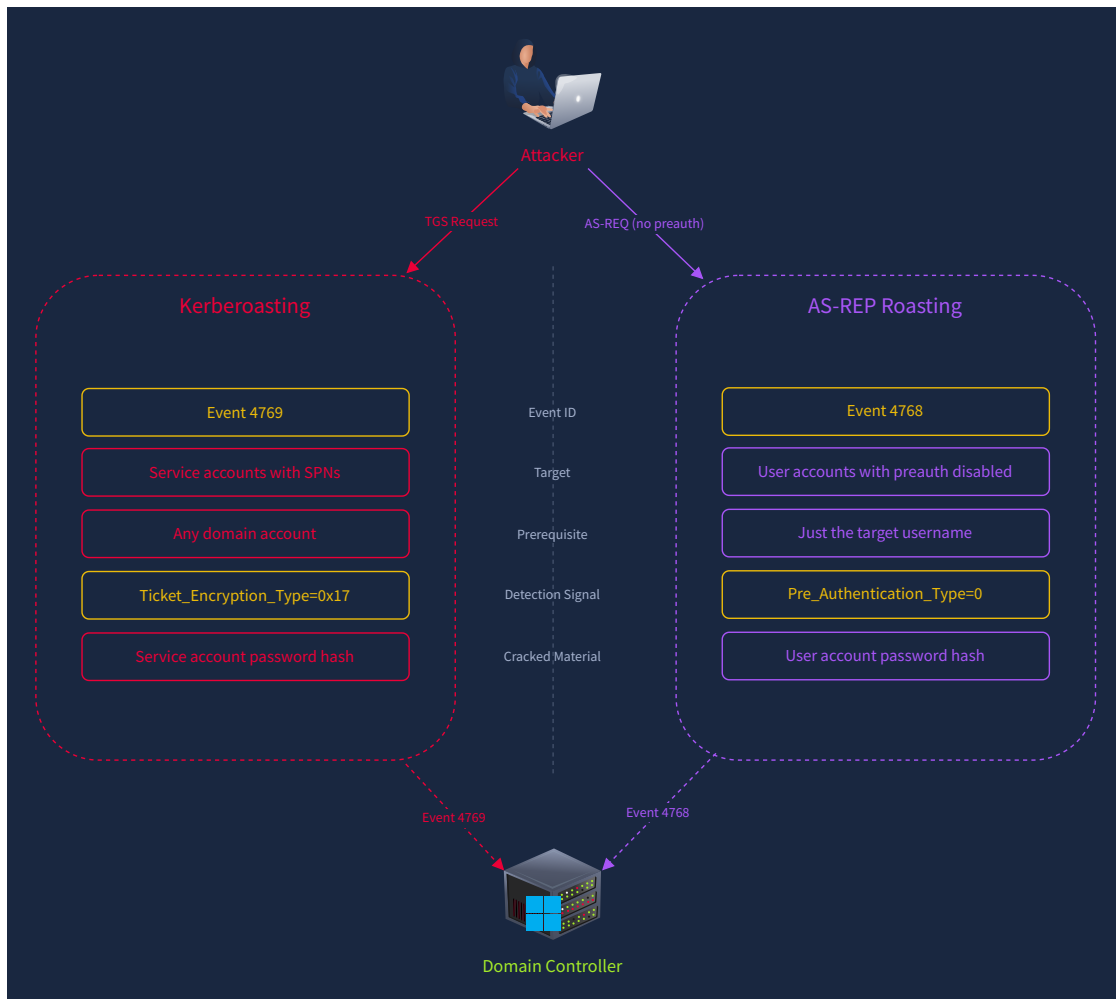
```
index=task3 (EventCode=4624 OR EventCode=4769)
| search Account_Name="{ACCOUNT_NAME}"
| table _time, EventCode, Account_Name, Client_Address
```

The screenshot shows a Splunk search interface. The search bar contains the query: `index=task3 (EventCode=4624 OR EventCode=4769) | search Account_Name="{ACCOUNT_NAME}" | table _time, EventCode, Account_Name, Client_Address`. Below the search bar, it indicates "0 events (before 3/16/26 2:18:28.000 AM)". The interface also shows tabs for Events, Patterns, Statistics (0), and Visualization. At the bottom, it states "No results found."

The query returns zero results. That strongly suggests the TGT was requested for offline cracking rather than for normal service access, though it is not definitive by itself. A legitimate application could still produce this pattern if it failed before requesting a service ticket, relied on a cached ticket, or otherwise did not complete the expected Kerberos flow.

Even so, when the same event also shows `Pre_Authentication_Type=0` and RC4 encryption, the absence of follow-up activity makes malicious intent much more likely.

Kerberoasting vs AS-REP Roasting



Answer the questions below

Which account had preauthentication disabled? (Answer Format: username)

alex.reed

```
index=task3 EventCode=4768 Pre_Authentication_Type=0
| table _time, Account_Name, Pre_Authentication_Type, Ticket_Encryption_Type, Client_Address
```

The screenshot shows the Splunk Enterprise interface with a search query: `index=task3 EventCode=4768 Pre_Authentication_Type=0`. The results table has the following columns: `_time`, `Account_Name`, `Pre_Authentication_Type`, `Ticket_Encryption_Type`, and `Client_Address`. Two events are displayed, both for the user `alex.reed` at `2026-03-15 02:06:42`.

_time	Account_Name	Pre_Authentication_Type	Ticket_Encryption_Type	Client_Address
2026-03-15 02:06:42	alex.reed	0	0x17	::ffff:10.5.90.1
2026-03-15 02:06:42	alex.reed	0	0x17	::ffff:10.5.90.1

Task 4 - Detecting LSASS Credential Dumping

LSASS Credential Dumping is a post-exploitation cyber attack technique where an attacker extracts plaintext passwords, NTLM hashes, Kerberos tickets, and user tokens directly from the memory of the **Local Security Authority Subsystem Service (LSASS)** process in Windows.

lsass.exe (Local Security Authority Subsystem Service) is a critical core Windows process responsible for enforcing security policy, authenticating user logons, creating access tokens, and writing entries to the Windows Security Log.

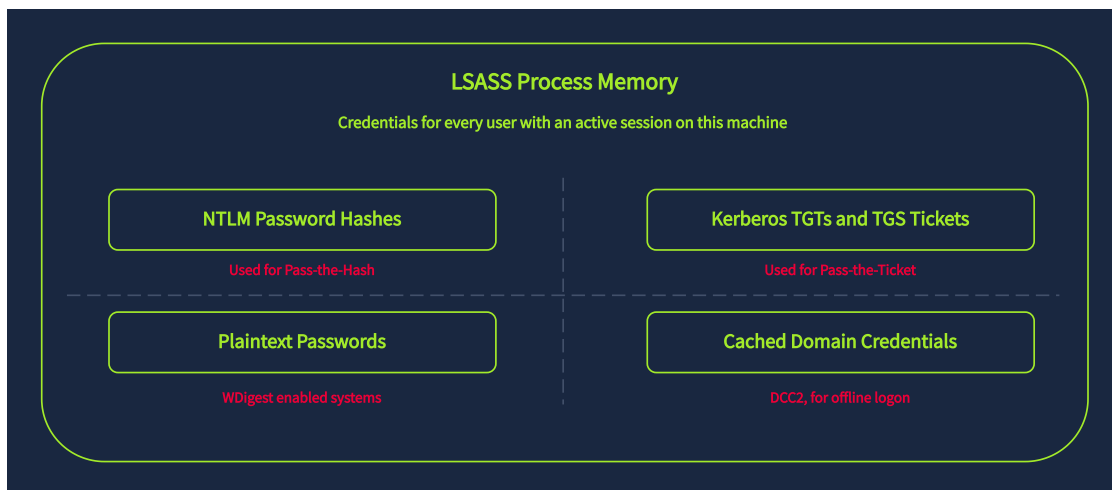
Attackers target LSASS (Local Security Authority Subsystem Service) because it stores credentials for every user who has authenticated to the machine. If a domain admin logged into a workstation earlier that day, their NTLM hash and Kerberos tickets are sitting in LSASS memory. An attacker who dumps LSASS on that machine gets those credentials immediately, no cracking required.

Let's think like an attacker. We've Kerberoasted some service accounts in the previous task, but none had domain admin privileges. **So what would we do next?** There are different approaches attackers can take, but one of them is finding a machine where a DA or any account with higher privileges has an active session. Dumping LSASS is the next escalation path.

What LSASS Stores: LSASS holds different types of credentials depending on the Windows version and configuration:

- NTLM (NT LAN Manager) password hashes for all authenticated users
- Kerberos tickets (TGTs and TGS tickets) for active sessions
- Plaintext passwords on systems where WDigest is enabled (Windows 8/Server 2012 and earlier by default, or any newer version where the `UseLogonCredential` registry value has been set to `1`)
- Cached domain credentials for offline logon

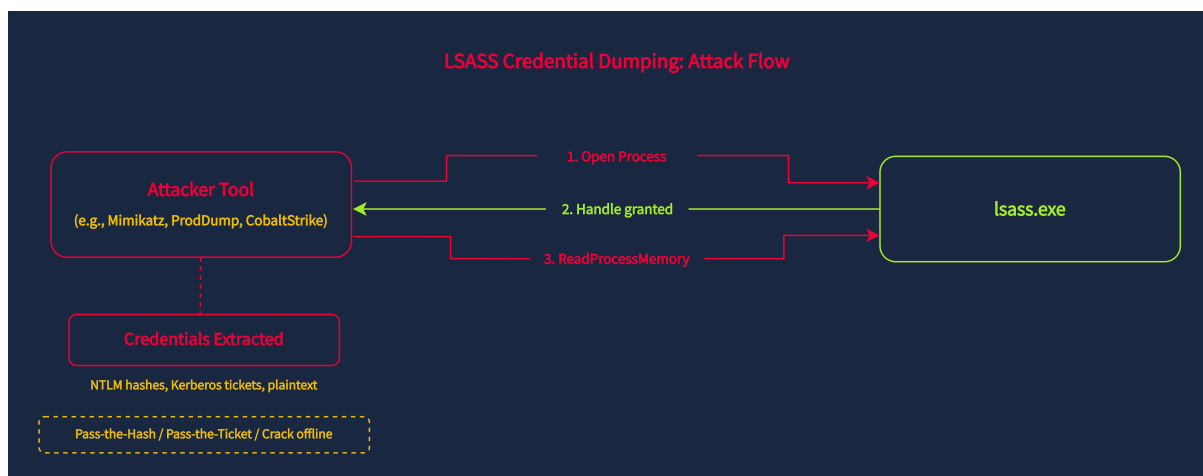
The diagram below shows the different credential types stored in LSASS memory:



WDigest (Windows Digest Authentication) is a legacy security protocol introduced in Windows XP to support HTTP Digest and SASL authentication.

DCC2 (Domain Cached Credentials version 2), also known as **MSCache2** or **MS-Cache v2** is a hashing format used by Windows (Vista, Server 2008, and newer) to store a user's domain password locally.

If an attacker dumps LSASS, they can use NTLM hashes to crack them offline, use them in Pass-the-Hash attacks, or dump Kerberos tickets for Pass-the-Ticket attacks. These are lateral movement techniques with their own detection artifacts. For now, our focus is on detecting the dump itself. The diagram below illustrates the LSASS dumping attack flow:



Detection With Sysmon Event 10: Sysmon Event 10 (ProcessAccess) fires when one process opens a handle to another process. When a tool like Mimikatz or ProcDump

accesses `lsass.exe`, Sysmon records exactly which process did it, what level of access it requested, and the call stack that led to the access.

Warning: Sysmon Event 10 only logs process access when the Sysmon configuration includes explicit `ProcessAccess` rules targeting `lsass.exe`. The default Sysmon installation does NOT log these events. If your environment uses a minimal Sysmon config, LSASS dumping will be invisible to Sysmon.

Here are the fields we'll use for detection:

Field	What It Contains	Why It Matters
SourceImage	Full path of the process accessing LSASS	Identifies the tool used for the dump
SourceUser	The user account running the source process	Identifies the compromised account. SYSTEM is expected; a domain user account is suspicious
TargetImage	Full path of the target process (lsass.exe)	Confirms LSASS was the target
GrantedAccess	Hex access mask showing requested permissions	Different tools request different access levels
CallTrace	DLL call stack leading to the access	Reveals the method used (MiniDump API, injection, etc.)

Understanding the GrantedAccess Field: The `GrantedAccess` value is a hex bitmask built by adding together. Understanding how these values are composed helps when you encounter an unfamiliar access mask in the field:

Access Right	Hex Value	Purpose
PROCESS_QUERY_LIMITED_INFORMATION	0x1000	Query basic process info
PROCESS_QUERY_INFORMATION	0x0400	Query detailed process info
PROCESS_VM_READ	0x0010	Read process memory
PROCESS_ALL_ACCESS	0x1FFFFFFF	Full access to the process

For example, `0x1010` = `0x1000` (PROCESS_QUERY_LIMITED_INFORMATION) + `0x0010` (PROCESS_VM_READ). The `0x0010` bit is what matters for credential dumping, because reading LSASS process memory is how credentials are extracted. In practice, `0x1010` is associated with Mimikatz, and `0x1FFFFFFF` (PROCESS_ALL_ACCESS) with ProcDump, comsvcs.dll, and Task Manager.

Understanding the CallTrace Field: The `CallTrace` field shows the chain of DLL calls that led to the LSASS access, and it can distinguish between different dump methods:

- **Known DLLs** like `dbgcore.dll` and `dbghelp.dll` indicate the MiniDump API was used. This is how ProcDump and `comsvcs.dll` create memory dumps. It's a legitimate API being used

for a malicious purpose.

- **UNKNOWN memory offsets** (addresses not mapped to any known DLL) indicate injected code. This is the signature of Cobalt Strike beacons, Meterpreter, and other in-memory implants that access LSASS from injected shellcode.

To make this concrete, here's what each pattern looks like in a real CallTrace value:

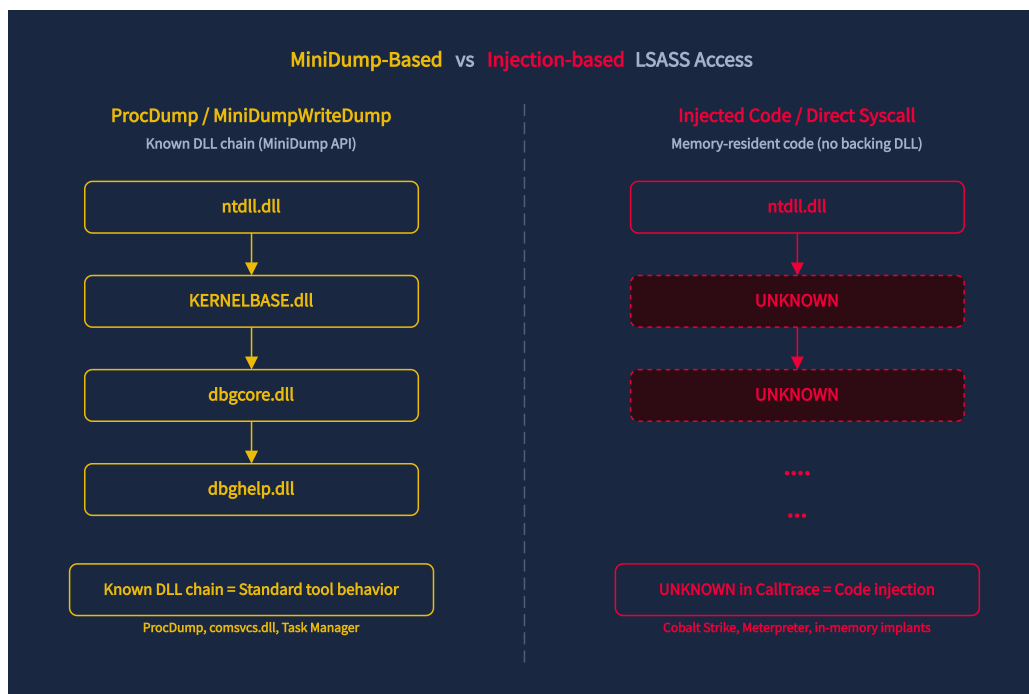
MiniDump-based (ProcDump/comsvcs.dll):

```
C:\Windows\SYSTEM32\ntdll.dll+9D4C4|C:\Windows\System32\KERNELBASE.dll+2B16D|C:\Windows\System32\dbgcore.dll+A3C8|...
```

Injection-based (e.g., Cobalt Strike):

```
C:\Windows\SYSTEM32\ntdll.dll+9D4C4|UNKNOWN(0000025FA0120000)|UNKNOWN(0000025FA0124B30)|...
```

The diagram below provides a visual comparison of these two CallTrace patterns:



The distinction matters because it tells you what kind of attacker you're dealing with. A ProcDump-based dump suggests a hands-on-keyboard attacker using LOLBins (legitimate tools for malicious purposes). An injection-based dump suggests a more sophisticated implant.

What Normal Looks Like: Several legitimate processes access LSASS as part of normal Windows operations. Knowing these helps us filter noise:

Full Process Path	Typical GrantedAccess	Why
C:\Windows\System32\csrss.exe	0x1000 or 0x1400	Windows subsystem, manages processes
C:\Windows\System32\WerFault.exe	0x1000	Windows Error Reporting crash handler
C:\Windows\System32\svchost.exe	0x1010	Various service functions (normal at this access level)
AV/EDR agent paths	Varies	Security products monitor LSASS for protection

Note: When filtering out legitimate processes during investigation, always match on the full path, not just the executable name. An attacker can name their tool `svchost.exe` and place it in `C:\Users\Public\` or `C:\Temp\`. The process name looks legitimate, but the path gives it away. The same logic applies to `csrss.exe`, `WerFault.exe`, and any other system process. If the `SourceImage` path isn't `C:\Windows\System32\`, treat it as suspicious regardless of the file name. This also means that access masks that are associated with credential dumping tools aren't automatically malicious. It all depends on context: which process is accessing LSASS, and why.

Walkthrough: Rather than jumping straight to known-bad access masks, we first want to see every process that accessed LSASS so we don't miss anything by filtering too early:

```
index=task4 EventCode=10 TargetImage="*\\lsass.exe"
| stats count by SourceImage, GrantedAccess
```

SourceImage	GrantedAccess	count
<REDACTED>.exe	0x??????	2
C:\Windows\system32\svchost.exe	0x1000	100
C:\Windows\system32\svchost.exe	0x101001	4

If we look at the results, we can see a mix of legitimate system processes and a suspicious entry. Once we identify the suspicious process, let's examine its CallTrace to understand the method used.

```
index=task4 EventCode=10 TargetImage="*\\lsass.exe" SourceImage={SUSPICIOUS_PROCESS}
```

| `table _time, SourceImage, SourceUser, GrantedAccess, CallTrace`

SourceImage	GrantedAccess	CallTrace
<REDACTED>.exe	0x??????	C:\Windows\SYSTEM32\ntdll.dll+9f234 C:\Windows\SYSTEM32\ntdll.dll+b043f C:\Windows\System32\KERNEL32.DLL+1845c C:\Windows\S
<REDACTED>.dll		C:\Windows\SYSTEM32\<REDACTED>.dll C:\Windows\SYSTEM32\<REDACTED>.dll C:\Windows\SYSTEM32\<REDACTED>.dll C:\
<REDACTED>.exe		<REDACTED>.exe <REDACTED>.exe <REDACTED>.exe C:\Windows\System32\KERNEL32.DLL+1244d C:\Windows\SYSTEM32\ntdll.dll+5df78
<REDACTED>.exe	0x??????	C:\Windows\SYSTEM32\ntdll.dll+9f234 C:\Windows\System32\KERNELBASE.dll+3212e <REDACTED>.exe C:\Windows\System32\KERNEL32.DL

GrantedAccess and **CallTrace** are two critical metadata fields captured primarily in **Sysmon Event ID 10** ("ProcessAccess"), as well as Windows Security Object Access auditing (Event IDs 4656/4663).

GrantedAccess is a hexadecimal bitmask value representing the specific permissions a source process requested (and received) when opening a handle to a target process.

Common Bitmask Permissions:

- `0x0010` (`PROCESS_VM_READ`): Allows reading the target process's memory space.
- `0x0020` (`PROCESS_VM_WRITE`): Allows writing into the process's memory space.
- `0x1410` / `0x1010` : Standard read access masks requested by credential dumping tools like **Mimikatz** (`sekurlsa::logonpasswords`).
- `0x1FFFFFF` (`PROCESS_ALL_ACCESS`): Full control over the target process (common in dumping utilities like `Procdump`).

CallTrace provides the stack trace of execution at the moment the source process opened the handle to the target process. It logs the sequence of DLLs, system functions, and memory addresses that executed the request. It reveals *how* the process made the call and helps differentiate legitimate software from malware.

The `CallTrace` tells us whether this was a MiniDump-based dump (known DLLs like `dbgcore.dll`) or an injection-based dump (`UNKNOWN` offsets). This distinction matters for the investigation because it changes what you look for next. The `SourceUser` field tells us which account was running the dump tool. If it's `NT AUTHORITY\SYSTEM` , the tool was executed with system-level privileges (possibly through a service or scheduled task). If it's a domain user account like `DOMAIN\jsmith` , that account is confirmed compromised and needs immediate investigation. Knowing the compromised account is a critical part of scoping the incident because it tells you which credentials, sessions, and systems that account had access to. In the BlackSuit intrusion, the attackers accessed LSASS through an injected `mstsc.exe` process with an access mask of `0x1010` and through `dllhost.exe` with `0x1FFFFFF` . Both processes had been injected with Cobalt Strike beacons, and the `CallTrace` contained `UNKNOWN` offsets rather than legitimate DLLs.

`dbgcore.dll` (Windows Core Debugging Helpers) is a legitimate, built-in Windows system library that provides API functions for generating process memory crash dumps.

`mstsc.exe` (**Microsoft Terminal Services Client**) is the built-in Windows executable responsible for initiating Remote Desktop Protocol (RDP) connections to remote computers, servers, and virtual machines.

Other Credential Stores: LSASS is not the only credential store on a Windows machine.

- The **SAM database** (`C:\Windows\System32\config\SAM`) stores local account hashes only, not domain credentials. Attackers extract SAM hashes with tools like `reg save` or Mimikatz's `lsadump::sam` , but these only give access to local accounts on that specific machine.
- **Cached domain credentials** (stored in the registry under `SECURITY\Cache`) are DCC2 hashes that allow offline logon when the DC is unreachable. These are slow to crack (bcrypt-based) and can't be used for Pass-the-Hash, making them a lower-priority target.
- The **NTDS.dit** database on domain controllers contains every account hash in the domain. Extracting it requires privileged access on the DC itself, which we cover in Task 6.

LSASS is the primary target because it holds live domain credentials in a directly usable format. If an attacker dumps LSASS on a machine where a Domain Admin has an active session, they can immediately use those credentials for Pass-the-Hash or Pass-the-Ticket without any cracking.

Answer the questions below

| What is the full path of the process that accessed lsass.exe?

C:\Windows\Temp\procdump64.exe

| What GrantedAccess value was used? (Answer Format: 0xNNNNNNN)

0x1FFFFFF

New Search Save As Create Table View Close

1 index=task4 EventCode=10 TargetImage="*\\lsass.exe"
 2 | stats count by SourceImage, GrantedAccess

✓ 106 events (before 8/18/26 8:16:10.000 AM) No Event Sampling Job || [Icons] Smart Mode

Events Patterns **Statistics (3)** Visualization

Show: 20 Per Page Format Preview: On

SourceImage	GrantedAccess	count
C:\Windows\Temp\procdump64.exe	0x1FFFFFF	2
C:\Windows\system32\svchost.exe	0x1000	100
C:\Windows\system32\svchost.exe	0x101001	4

Which DLL in the CallTrace reveals the dump method?

dbgcore.dll

Task 5 - Detecting DCSync

DCSync is a malicious technique used by attackers to impersonate a **Domain Controller** and extract password hashes, Kerberos keys, and other sensitive credentials from **Active Directory** without directly accessing the target machine's memory or the NTDS.dit file.

NTDS.dit (NT Directory Services - Directory Information Tree) is the primary database file used by Active Directory Domain Controllers to store all domain objects, user accounts, password hashes, group memberships, and Kerberos keys.

At this point in the attack chain, imagine the attacker has Domain Admin credentials. Maybe they cracked a Kerberoasted service account that happened to be a Domain Admin, or they dumped LSASS on a machine where a Domain Admin had an active session. Either way, they now have the highest level of access in the domain.

Attackers target DCSync because it lets them extract every password hash in the domain without touching the domain controller's disk, without creating volume shadow copies, and without needing physical access to the DC. It works by abusing the Active Directory replication protocol (DRSUAPI), the same protocol that legitimate domain controllers use to synchronize directory data with each other. The attacker's machine pretends to be a domain controller and requests password data via DRSUAPI. If their account has the right permissions, the real DC complies. Domain Admins, Enterprise Admins, and DC machine accounts have these replication rights by default.

DRSUAPI (**Directory Replication Service Remote Protocol**, specification **MS-DRSR**) is an RPC-based interface used by Active Directory Domain Controllers to replicate directory data—

such as user accounts, group memberships, and credential hashes—across a domain or forest.

RPC-based (Remote Procedure Call) describes any system, protocol, or software architecture that relies on **RPC** to execute code or request services on a remote computer across a network as if the function call were local.

In some environments, attackers can also obtain replication rights through ACL abuse (WriteDACL on the domain object) without ever joining the Domain Admins group.

ACL abuse (Access Control List abuse) in Active Directory (AD) refers to techniques where an attacker exploits misconfigured rights and permissions assigned to domain objects (users, groups, computers, or OUs) to execute domain privilege escalation, stealthy persistence, or lateral movement.

WriteDACL (Write Discretionary Access Control List) is a high-risk Active Directory permission that gives a security principal (user, group, or computer) the ability to modify the access rights (DACL) assigned to a target AD object.

DACL (Discretionary Access Control List), which directly connects to the Active Directory permission concepts you've been exploring (like **WriteDACL** and **ACL abuse**).

How DCSync Works: Active Directory replication is built on a set of extended rights that control who can request directory data. Three specific GUIDs (A **GUID (Globally Unique Identifier)** is a 128-bit number used in software applications, operating systems, and databases to uniquely identify resources without requiring a central authority to coordinate generation) are relevant:

- **{1131f6ad-9c07-11d1-f79f-00c04fc2dcd2}** is DS-Replication-Get-Changes-All
- **{1131f6aa-9c07-11d1-f79f-00c04fc2dcd2}** is DS-Replication-Get-Changes
- **{89e95b76-444d-4c62-991a-0facbeda640c}** is DS-Replication-Get-Changes-In-Filtered-Set

GUID	Display Name	Functional Description
1131f6aa-9c07-11d1-f79f-00c04fc2dcd2	Replicating Directory Changes	Grants permission to read/replicate standard Active Directory directory changes.
1131f6ad-9c07-11d1-f79f-00c04fc2dcd2	Replicating Directory Changes All	Grants permission to replicate secret data —including password hashes (NTLM), Kerberos keys, and LSA secrets.
89e95b76-444d-4c62-991a-0facbeda640c	Replicating Directory Changes In Filtered Set	Grants permission to replicate attributes marked in the Filtered Attribute Set (PAS) —used primarily by Read-Only Domain Controllers (RODCs).

These three strings are the exact **Active Directory Extended Right GUIDs** required to perform directory replication across Active Directory Domain Controllers. Together, they

form the permission set that enables **DCSync attacks**.

The one that matters most is `1131f6ad`, which is DS-Replication-Get-Changes-All. This is the permission that allows pulling password data, and it's the primary indicator of DCSync.

Detection with Event 4662: DCSync detection uses Event 4662 (An operation was performed on an object), which is part of Directory Service Access auditing. When someone exercises the replication extended rights on the domain partition, Event 4662 fires with the replication GUID embedded in the raw event data.

Field	What It Contains	Why It Matters
user	The account performing the replication	Identifies who is running DCSync
Access_Mask	0x100 (Control Access)	Indicates an extended right was exercised
Properties	Shows "Control Access" (GUIDs in raw event)	The replication GUIDs confirm DCSync
Logon_ID	Hex session identifier	Links to 4624 logon event for source IP correlation

The detection signal is an Event 4662 with `Access_Mask=0x100`, the replication GUIDs present in the raw event data, and a `user` that isn't a machine account (doesn't end in `$`).

▲

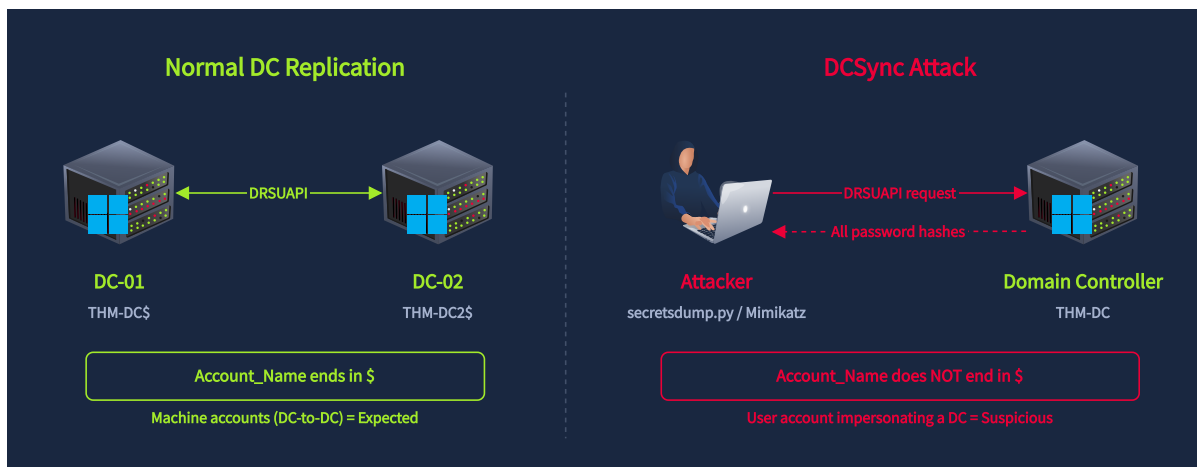
Warning: Event 4662 for DCSync detection requires TWO things to be configured in advance: (1) "Audit Directory Service Access" must be enabled via Group Policy, and (2) a SACL (System Access Control List) must be set on the domain partition to audit replication operations. Neither is enabled by default. Without both of these, DCSync is completely invisible in the logs. This is one of the most common detection gaps in real environments, and it's worth verifying in any network you're responsible for.

ACL (Access Control List): The overarching term for permission/audit lists in Windows.

DACL (Discretionary ACL): Controls permissions (who can access or modify the object).

SACL (System ACL): Controls auditing (which actions on the object trigger Windows Security logs).

Normal vs Suspicious Replication: In a production environment with multiple domain controllers, Event 4662 events with replication GUIDs are completely normal. Domain controllers replicate constantly. The distinction between normal and malicious is the source, as shown in the diagram below:



Pattern	Normal	Suspicious
user	Machine account ending in \$ (e.g., THM-DC\$)	Human user account (not ending in \$)
Source host	Another domain controller	A workstation or non-DC server
Frequency	Regular intervals matching replication schedule	One-time or burst of requests
Scope	Specific partition changes	Requesting all credentials (-just-dc-ntlm or full dump)

Walkthrough: With the normal replication patterns from the table above as our baseline, we can filter for the anomaly: a non-machine account exercising replication rights.

The `1131f6ad` GUID (DS-Replication-Get-Changes-All) is the permission that allows pulling password data, and filtering out machine accounts (`user!="*$"`) isolates any human user performing replication:

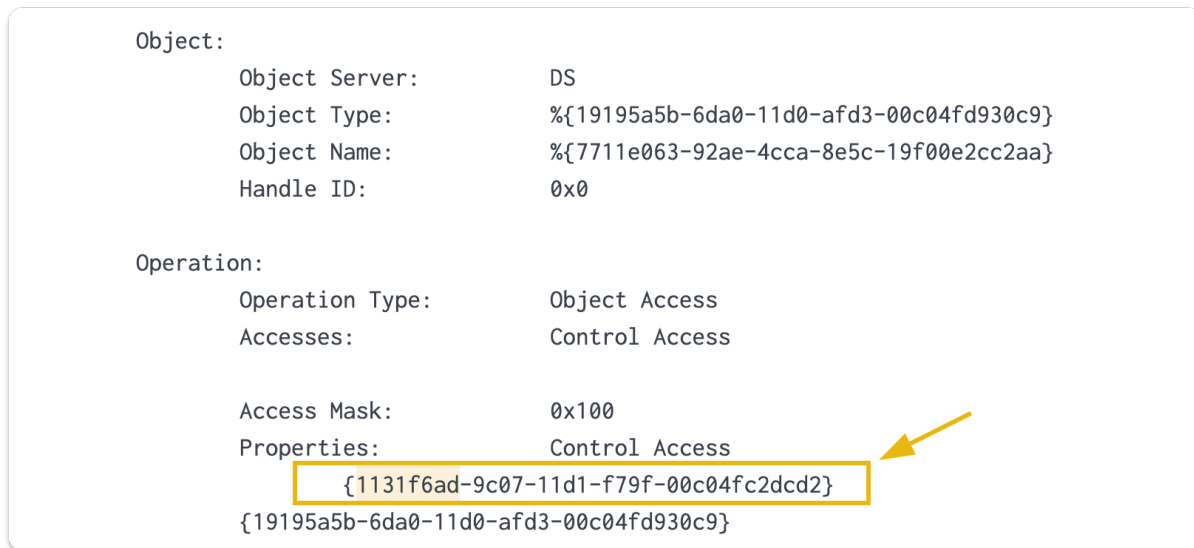
```
index=task5 EventCode=4662 "1131f6ad" user!="*$"
| table _time, user, Access_Mask, Properties
| sort _time
```

_time	user	Access_Mask	Properties
2026-03-17 05:30:47	<REDACTED>	0x100	Control Access

This query returned results, which means there's a non-machine account performing AD replication. The `user` field tells us which account was used, and the `Properties` field confirms this was a Control Access operation. The replication GUID `1131f6ad` that we filtered for is

embedded in the raw event data, which is why we search for it as a keyword rather than a field filter.

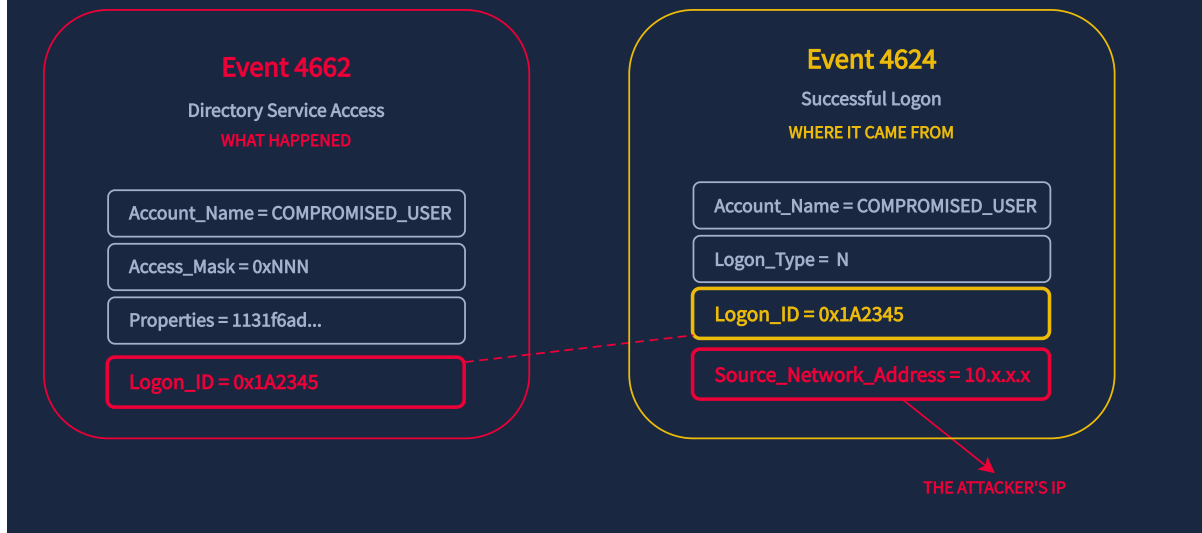
If you click on the event to expand it, you'll see the full GUID `{1131f6ad-9c07-11d1-f79f-00c04fc2dcd2}` in the raw log details under the Properties section. Splunk's field extraction parses this as "Control Access" (the access type) but doesn't extract the individual GUIDs into separate fields. That's why our query uses `"1131f6ad"` as a keyword search against the raw event text rather than filtering on the `Properties` field directly.



Warning: The `user!="*$"` filter assumes attackers use human accounts for DCSync. In practice, adversaries who compromise machine account hashes (through ADCS abuse, unconstrained delegation, or other exploits) can perform DCSync from a machine account. In those cases, the `$` filter would hide the attack. For mature environments, consider monitoring ALL accounts performing replication and baselining which machine accounts normally replicate.

Getting the Source IP: Event 4662 doesn't contain a source IP field directly. To find where the DCSync originated, we correlate the `Logon_ID` from the 4662 event with a 4624 (logon) event for the same session, which contains the `Source_Network_Address` field. The diagram below illustrates this correlation:

Finding The Source IP



```
index=task5 EventCode=4662 Access_Mask=0x100 user={COMPROMISED_USER}
"1131f6ad"
| table _time, host, user, Logon_ID
```

_time	host	user	Logon_ID
2026-03-17 05:30:47	THM-DC	<REDACTED>	<REDACTED>

Access_Mask: 0x100 (Control Access) > Indicates an extended right was exercised

```
index=task5 EventCode=4624 Logon_ID={LOGON_ID}
| table _time, host, user, Source_Network_Address, Logon_Type
```

_time	host	user	Source_Network_Address	Logon_Type
2026-03-17 05:30:47	THM-DC	<REDACTED>	<REDACTED>	<REDACTED>

This gives us the source IP and confirms the logon type. **Source_Network_Address** is an essential metadata field found inside Windows Security Logs (e.g., Event IDs 4624 and 4625) and endpoint telemetry (Sysmon).

Answer the questions below

| What account performed the DCSync?

adm-luke.sullivan

1 index=task5 EventCode=4662 "1131f6ad" user!="*\$"

2 | table _time, user, Access_Mask, Properties

3 | sort _time

✓ 1 event (before 8/18/26 8:29:38.000 AM) No Event Sampling ▾

Events (1) Patterns Statistics (1) Visualization

Timeline format ▾ — Zoom Out + Zoom to Selection × Deselect

Format ▾ Show: 20 Per Page ▾ View: List ▾

< Hide Fields

≡ All Fields

SELECTED FIELDS

a host 1

a source 1

a sourcetype 1

INTERESTING FIELDS

a Access_Mask 1

a Accesses 1

a Account_Domain 1

a Account_Name 1

a action 1

a app 1

a body 1

a category 1

a ComputerName 1

a dest 1

i

Time

Event

> 3/17/26 03/16/2026 10:30:47 PM

5:30:47.000 AM LogName=Security

EventCode=4662

EventType=0

Account_Name

1 Value, 100% of events

Selected Yes No

Reports

Top values Top values by time Rare values

Events with this field

Values	Count	%
adm-luke.sullivan	1	100%

What is the Logon_ID of the DCSync session? (Answer Format: 0xNNNNNNNN)

0x5A01668

i	Time	Event
>	3/17/26 5:30:47.000 AM	03/16/2026 10:30:47 PM LogName=Security EventCode=4662 EventType=0 ComputerName=THM-DC.tryhatmestudios.thm SourceName=Microsoft Windows security auditing. Type=Information RecordNumber=565670 Keywords=Audit Success TaskCategory=Directory Service Access OpCode=Info Message=An operation was performed on an object. Subject : Security ID: S-1-5-21-850000725-1200463294-3662097055-1120 Account Name: adm-luke.sullivan Account Domain: tryhatmestudios Logon ID: 0x5A01668 Object: Object Server: DS Object Type: %{19195a5b-6da0-11d0-afd3-00c04fd930c9} Object Name: %{7711e063-92ae-4cca-8e5c-19f00e2cc2aa} Handle ID: 0x0 Operation: Operation Type: Object Access Accesses: Control Access Access Mask: 0x100 Properties: Control Access

What source IP initiated the DCSync?

10.5.90.1

Task 6 - Detecting NTDS.dit Extraction

DCSync works remotely and blends in with replication traffic. But it's not the only way attackers extract credentials from Active Directory. Before DCSync became the standard, attackers copied the **NTDS.dit** file directly from the domain controller. This older approach is noisier, but it's still actively used in real intrusions.

The **NTDS.dit** file is the Active Directory database stored on every domain controller (typically at **C:\Windows\NTDS\ntds.dit**). It contains password hashes for every account in the domain. The problem for attackers is that Windows locks this file while AD DS (Active Directory Domain Services - is the core server role in Microsoft Windows Server that turns a server into a **Domain Controller**) is running, so it can't be copied directly. Attackers use two main workarounds:

Volume Shadow Copy (vssadmin): Creates a point-in-time snapshot of the filesystem, bypassing the file lock. The attacker creates a shadow copy, copies `ntds.dit` and the `SYSTEM` registry hive (needed to decrypt the hashes) from the snapshot, then deletes the shadow copy.

Typical command sequence:

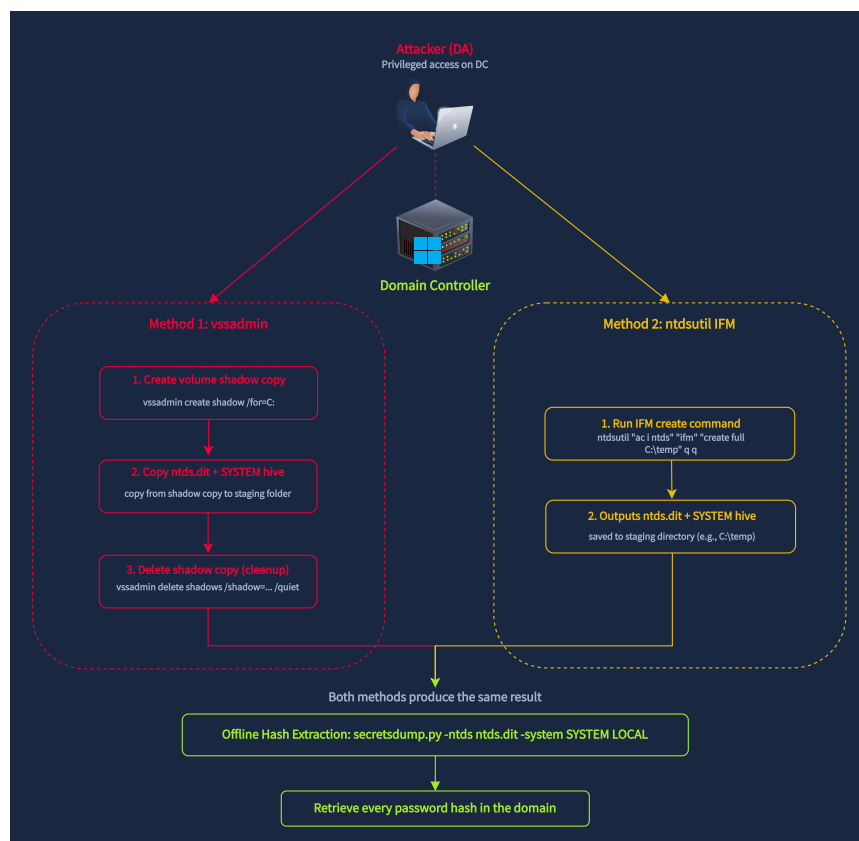
```
C:\Windows\System32> vssadmin create shadow /for=C:
C:\Windows\System32> copy \\?\GLOBALROOT\Device\HarddiskVolumeShadowC
opy1\Windows\NTDS\ntds.dit C:\temp\ntds.dit
C:\Windows\System32> copy \\?\GLOBALROOT\Device\HarddiskVolumeShadowC
opy1\Windows\System32\config\SYSTEM C:\temp\SYSTEM
C:\Windows\System32> vssadmin delete shadows /shadow={shadow-id} /qui
et
```

Install From Media / ntdsutil: A legitimate AD management tool designed for domain controller promotion. Attackers abuse the IFM (Install From Media) feature to produce a clean copy of `ntds.dit` and the `SYSTEM` hive, saving them to a local directory.

Typical command:

```
C:\Windows\System32> ntdsutil "ac i ntds" "ifm" "create full C:\temp"
q q
```

Both methods require privileged access on the domain controller. `vssadmin` needs local admin rights, while `ntdsutil` requires Domain Admin or equivalent AD DS permissions. Both produce the same result: an offline copy of the AD database that can be parsed with tools like `secretsdump.py` or `NTDSDumpEx` to extract all password hashes. The diagram below shows both extraction paths:



Detecting NTDS.dit Extraction: NTDS.dit extraction is detected through process creation events. We're looking for `vssadmin.exe` or `ntdsutil.exe` being executed on a domain controller with command-line arguments related to shadow copies or IFM.

`vssadmin.exe` (**Volume Shadow Copy Service Administrative Command-Line Tool**) is a native Windows binary located in `C:\Windows\System32\`. It manages system restore points, volume shadow copies, and shadow storage allocations.

`ntdsutil.exe` is a built-in command-line tool in Windows Server located in `C:\Windows\System32\` that provides advanced management database utilities for **Active Directory Domain Services (AD DS)**.

Note: If Sysmon isn't deployed on your domain controllers, Windows Security Event 4688 (Process Creation) with command-line logging enabled provides the same process and command-line visibility. The queries in this walkthrough use Sysmon Event 1, but the detection logic also applies to 4688.

Sysmon Event 1 (Process Creation) captures the full command line. The detection signals are:

- `ntdsutil.exe` with a command line containing `ifm` and `create`

- `vssadmin.exe` with a command line containing `create shadow`

Sysmon Event 11 (File Creation) captures when `ntds.dit` is written to an unusual location by `ntdsutil.exe`.

`create shadow` is the command-line flag used with `vssadmin.exe` or PowerShell to manually instantiate a new Volume Shadow Copy for a specified target drive volume in Windows. It generates a point-in-time, read-only snapshot of an entire disk volume, capturing the file state at the exact moment of execution.

Walkthrough 1: Investigating ntdsutil Malicious Use: We start by looking for `ntdsutil.exe` execution because the IFM subcommand is the most common extraction method, and Sysmon Event 1 captures the full command line:

```
index=task6 EventCode=1 Image="*\ntdsutil.exe"
| table _time, host, User, ParentImage, Image, CommandLine
```

_time	host	User	ParentImage	Image	CommandLine
2026-03-18 07:57:13	THM-DC	<REDACTED>	<REDACTED>	C:\Windows\System32\ntdsutil.exe	<REDACTED>

The `CommandLine` field shows the full `ntdsutil` command used. The `ParentImage` tells us how `ntdsutil` was launched.

Now let's check if the extraction produced the expected file:

```
index=task6 EventCode=11 TargetFilename="*\ntds.dit" Image="*\ntdsuti
l.exe"
| table _time, Image, TargetFilename
```

_time	Image	TargetFilename
2026-03-18 07:57:17	C:\Windows\system32\ntdsutil.exe	<REDACTED>

If this returns a result, the NTDS.dit file was successfully written to disk.

The `TargetFilename` shows where the attacker saved it. (most notably within **Sysmon (System Monitor)** and native **Windows Object Access (File System)** logs.)

Walkthrough 2: Investigating vssadmin Shadow Copy: The second extraction method uses `vssadmin` to create a volume shadow copy, then copies `ntds.dit` and the `SYSTEM` hive from the snapshot.

Let's search for the shadow copy creation:

```
index=task6 EventCode=1 Image="*\vssadmin.exe" CommandLine="*create s
hadow*"
| table _time, host, User, ParentImage, Image, CommandLine
```

_time	host	User	ParentImage	Image	CommandLine
2026-03-18 07:57:29	THM-DC	<REDACTED>	<REDACTED>	C:\Windows\System32\vssadmin.exe	<REDACTED>

The **CommandLine** confirms a shadow copy was created. Creating a shadow copy alone has legitimate uses (backups, system restore), so this event by itself doesn't confirm credential theft. What confirms it is what happens next.

Let's look for copy commands that target sensitive files from the shadow copy path:

```
index=task6 EventCode=1 CommandLine="*HarddiskVolumeShadowCopy*" (Com
mandLine="*ntds*" OR CommandLine="*SYSTEM*")
| table _time, host, User, ParentImage, Image, CommandLine
```

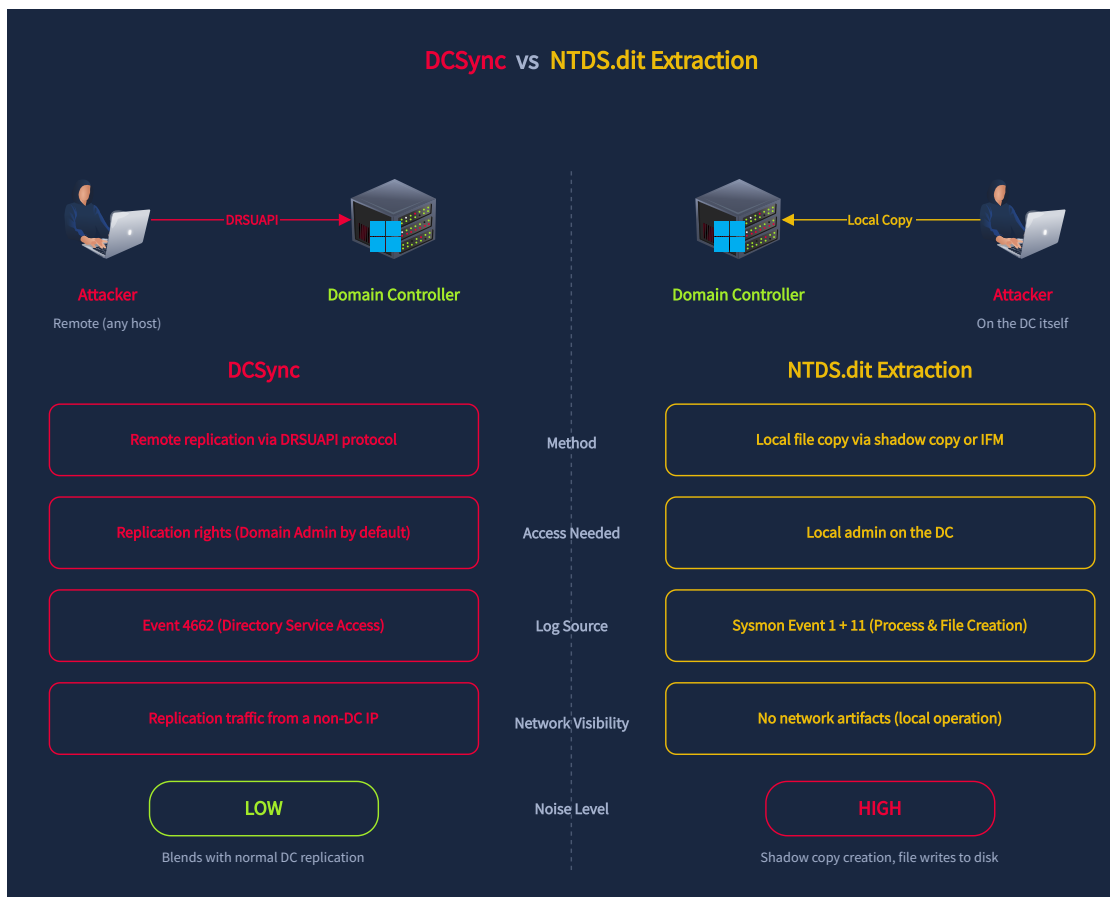
_time	host	User	ParentImage	Image	CommandLine
2026-03-18 07:57:42	THM-DC	<REDACTED>	<REDACTED>	C:\Windows\System32\cmd.exe	<REDACTED>
2026-03-18 07:57:40	THM-DC	<REDACTED>	<REDACTED>	C:\Windows\System32\cmd.exe	<REDACTED>

The **CommandLine** field reveals the full source and destination paths the attacker used. Both files together give the attacker everything needed to extract every credential in the domain offline.

DCSync vs NTDS.dit Extraction

Attribute	DCSync	NTDS.dit Extraction
Method	Remote replication via DRSUAPI	Local file copy via shadow copy or IFM
Access needed	Replication rights (DA by default)	Local admin on the DC
Log source	Event 4662 (Directory Service Access)	Sysmon Event 1 (Process Creation), Event 11 (File Creation)
Network visibility	Replication traffic from a non-DC IP	No network artifacts (local operation)
Noise level	Low (blends with normal replication)	High (shadow copy creation, file writes)

The diagram below visualizes these differences:



Answer the questions below

What is the full command line used to extract NTDS.dit? (Answer Format: full command line as shown in Splunk)

```
ntdsutil "ac i ntds" "ifm" "create full C:\Perflogs\1" q q
```

```
index=task6 EventCode=1 Image="*\ntdsutil.exe"
| table _time, host, User, ParentImage, Image, CommandLine
```

What is the full shadow copy path the attacker copied ntds.dit from? (Answer Format: full path as shown in the CommandLine)

```
CommandLine: cmd /c copy "\\?
\GLOBALROOT\Device\HarddiskVolumeShadowCopy7\Windows\System32\config\SYSTEM"
C:\Windows\Temp\system.save
```

```
index=task6 EventCode=1 CommandLine="HarddiskVolumeShadowCopy" (Comm
ndLine="ntds" OR CommandLine="SYSTEM")
```

| table _time, host, User, ParentImage, Image, CommandLine

New Search Save As Create Table View Close

1 index=task6 EventCode=1 CommandLine="*HarddiskVolumeShadowCopy*" (CommandLine="*ntds*" OR CommandLine="*SYSTEM*")
2 | table _time, host, User, ParentImage, Image, CommandLine

2 events (before 8/18/26 9:54:12.000 AM) No Event Sampling Job View Download Print Verbose Mode

Events (2) Patterns Statistics (2) Visualization

Timeline format Zoom Out Zoom to Selection Deselect 100 milliseconds per column

Format Show: 20 Per Page View: List

Time	Event
3/18/26 7:57:42.000 AM	03/18/2026 12:57:42 AM LogName=Microsoft-Windows-Sysmon/Operational EventCode=1 EventType=4 ComputerName=THM-DC.tryhatmestudios.thm User=NOT_TRANSLATED Sid=S-1-5-18 SidType=0 SourceName=Microsoft-Windows-Sysmon Type=Information RecordNumber=136762 Keywords=None TaskCategory=Process Create (rule: ProcessCreate) OpCode=Info Message=Process Create: RuleName: technique_id=T1059.003,technique_name=Windows Command Shell UtcTime: 2026-03-18 07:57:42.832 ProcessGuid: {0f65264c-5af6-69ba-9b7d-000000002000} ProcessId: 5704 Image: C:\Windows\System32\cmd.exe FileVersion: 10.0.20348.1 (WinBuild.160101.0800) Description: Windows Command Processor Product: Microsoft Windows® Operating System Company: Microsoft Corporation OriginalFileName: Cmd.Exe CommandLine: cmd /c copy "\\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy7\Windows\System32\config\SYSTEM" C:\Windows\Temp\system.save CurrentDirectory: C:\Users\adm-luke.sullivan\ User: tryhatmestudios\adm-luke.sullivan LogonGuid: {0f65264c-5af6-69ba-1ad2-810700000000}

SELECTED FIELDS
a host 1
a source 1
a sourcetype 1

INTERESTING FIELDS
a category 1
a CommandLine 2
a Company 1
a ComputerName 1
a CurrentDirectory 1
a Description 1
a dvc 1
a dvc_nt_host 1
event_id 2
EventCode 1
EventType 1
a eventtype 3
a FileVersion 1
a Hashes 1
id 2
a Image 1
a index 1
a IntegrityLevel 1
a Keywords 1
linecount 1
a LogName 1
a LogonGuid 2

Where did the attacker stage the files copied from the shadow copy? (Answer Format: directory path)

C:\Windows\Temp

```
CommandLine: cmd /c copy "\\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy7\Windows\System32\config\SYSTEM" C:\Windows\Temp\system.save
CurrentDirectory: C:\Users\adm-luke.sullivan\
User: tryhatmestudios\adm-luke.sullivan
LogonGuid: {0f65264c-5af6-69ba-1ad2-810700000000}
LogonId: 0x781D21A
TerminalSessionId: 0
IntegrityLevel: High
Hashes: SHA1=2ED89B5430C775306B316BA3A926D7DE4FE39FC7,MD5=E7A6B1F51EFB405287A8048CFA4690F4,SHA256=EB71EA69DD19F728AB9240565E8C7EFB59821E19E3788E289301E1E74940C208,IMPHASH=D60B77062898DC6BFAE7FE11A0F8806C
ParentProcessGuid: {0f65264c-5af6-69ba-997d-000000002000}
ParentProcessId: 768
ParentImage: C:\Windows\System32\cmd.exe
ParentCommandLine: "c:\windows\system32\cmd.exe" /c "cmd /c copy "\\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy7\Windows\System32\config\SYSTEM" C:\Windows\Temp\system.save"
ParentUser: tryhatmestudios\adm-luke.sullivan
```

[Collapse](#)

Task 7 - Investigation Challenge

A detection rule triggered on your SIEM for a burst of TGS requests with RC4 encryption from a single source within a 60-second window. The requests targeted multiple service accounts across different departments, and the source IP doesn't belong to any known service or scheduled task. Investigate whether this is a legitimate application or an active intrusion, and determine how far the attacker got.

Answer the questions below

Which account was targeted by AS-REP Roasting?

mia.turner

New Search

1 index=task7 Pre_Authentication_Type=0

✓ 2 events (before 8/18/26 9:58:15.000 AM) No Event Sampling ▼

Events (2) Patterns Statistics Visualization

Timeline format ▼ Zoom Out Zoom to Selection X Deselect

Format ▼ Show: 20 Per Page ▼ View: List ▼

< Hide Fields All Fields

SELECTED FIELDS

- host 1
- source 1
- sourcetype 1

INTERESTING FIELDS

- Account_Name 1
- action 1
- app 1
- body 2
- category 1
- Client_Address 1
- Client_Port 2
- ComputerName 1
- dest 1

Account_Name

1 Value, 100% of events

Selected Yes No

Reports

- Top values
- Top values by time
- Rare values
- Events with this field

Values	Count	%
mia.turner	2	100%

Show all 20 lines

host = THM-DC source = WinEventLog:Security sourcetype = WinEventLog

What account performed the Kerberoasting? (Answer Format: username only, without @domain)

```
index=task7 EventCode=4769 Ticket_Encryption_Type=0x17 Service_Name!=
"*$" Service_Name!="krbtgt"
```

New Search

1 `index=task7 EventCode=4769 Ticket_Encryption_Type=0x17 Service_Name!="*" Service_Name!="krbtgt"`

✓ 9 events (before 8/18/26 9:59:26.000 AM) No Event Sampling ▼

Events (9) Patterns Statistics Visualization

✓ Timeline format ▼ — Zoom Out + Zoom to Selection × Deselect

< Hide Fields

SELECTED FIELDS

- host 1
- source 1
- sourcetype 1

INTERESTING FIELDS

- Account_Domain 1
- Account_Name 1
- action 1
- app 1
- body 9
- category 1
- Client_Address 1
- Client_Port 9
- ComputerName 1

Time

Event

3/17/26 03/17/2026 09:07:13 AM

4:07:13.000 PM LogName=Security

Account_Name

1 Value, 100% of events

Selected Yes No

Reports

Top values Top values by time Rare values

Events with this field

Values

Values	Count	%
nathan.brooks@TRYHATMESTUDIOS.THM	9	100%

What process accessed LSASS on the workstation?

rundll32.exe

New Search

1 `index=task7 EventCode=10 TargetImage="*\\lsass.exe" dll calltrace 0x1FFFFFF`

✓ 1 event (before 8/18/26 10:05:35.000 AM) No Event Sampling ▼

Events (1) Patterns Statistics Visualization

✓ Timeline format ▼ — Zoom Out + Zoom to Selection × Deselect

< Hide Fields

SELECTED FIELDS

- host 1
- source 1
- sourcetype 1

INTERESTING FIELDS

- CallTrace 1
- category 1
- ComputerName 1
- dvc 1
- dvc_nt_host 1
- event_id 1
- EventCode 1
- EventType 1
- eventtype 3

Time

Event

3/17/26 4:07:32.000 PM

SourceImage: C:\Windows\system32\rundll32.exe

TargetImage: C:\Windows\system32\lsass.exe

GrantedAccess: 0x1FFFFFF

CallTrace: C:\Windows\SYSTEM32\ntdll.dll+9f234|C:\Windows\SYSTEM32\ntdll.dll+b043f|C:\Windows\System32\KERNEL32.DLL+1845c|C:\Windows\System32\KERNEL32.DLL+2364e|C:\Windows\SYSTEM32\dbgcore.DLL+aeef|C:\Windows\SYSTEM32\dbgcore.DLL+1ac05|C:\Windows\SYSTEM32\dbgcore.DLL+13cf0|C:\Windows\SYSTEM32\dbgcore.DLL+69da|C:\Windows\SYSTEM32\dbgcore.DLL+75e1|C:\Windows\System32\comsvcs.dll+27daa|C:\Windows\system32\rundll32.exe+3c6e|C:\Windows\system32\rundll32.exe+7f9f|C:\Windows\system32\rundll32.exe+149e|C:\Windows\System32\KERNEL32.DLL+1244d|C:\Windows\SYSTEM32\ntdll.dll+5df78

SourceUser: tryhatmestudios\adm-luke.sullivan

Show all 28 lines

host = THM-DEV-WS | source = WinEventLog:Microsoft-Windows-Sysmon/Operational | sourcetype = WinEventLog

What **GrantedAccess** value was used for the LSASS dump? (Answer Format: 0xNNNNNNN)

0x1FFFFF

What account performed the DCSync attack?

adm-luke.sullivan

The screenshot shows the Windows Event Viewer interface. The 'Events (13)' tab is selected. The 'SourceUser' field is highlighted in the 'Selected Fields' list. A pop-up window titled 'SourceUser' is open, showing a table of values. The table has columns for 'Values', 'Count', and '%'. The values are 'NT AUTHORITY\SYSTEM' (Count: 12, %: 92.308%) and 'tryhatemstudios\adm-luke.sullivan' (Count: 1, %: 7.692%). The 'tryhatemstudios\adm-luke.sullivan' entry is highlighted in blue. The pop-up window also has a 'Selected' button and a 'Yes' button.

Values	Count	%
NT AUTHORITY\SYSTEM	12	92.308%
tryhatemstudios\adm-luke.sullivan	1	7.692%

Task 8 - Conclusion

This room covered five credential attack techniques that bridge an attacker's initial foothold to full domain compromise, each detected through a different log source and event type.

Technique	Event ID	Detection Signal	Log Source	Privilege Required
Kerberoasting	Security 4769	Ticket_Encryption_Type=0x17, multiple SPNs from one account	DC Security Log	Any domain user
AS-REP Roasting	Security 4768	Pre_Authentication_Type=0, no follow-up authentication	DC Security Log	None (just a username)

Technique	Event ID	Detection Signal	Log Source	Privilege Required
LSASS Dumping	Sysmon 10	TargetImage=lsass.exe, suspicious GrantedAccess and CallTrace	Endpoint Sysmon	Local admin
DCSync	Security 4662	Access_Mask=0x100, replication GUIDs, non-machine account	DC Security Log	Domain Admin (replication rights)
NTDS.dit Extraction	Sysmon 1, 11	ntdsutil.exe or vssadmin.exe with IFM/shadow copy arguments	DC Sysmon	Local admin on DC

- Each technique in the escalation chain uses a different log source. Kerberos attacks show up on the DC Security log, LSASS dumping requires Sysmon on endpoints, and DCSync needs Directory Service Access auditing on the DC.
- The events themselves aren't rare. What makes them suspicious is context: RC4 in an AES environment, a non-DC account performing replication, or a high-privilege access mask on LSASS from an unexpected process.
- Logging must be configured before the attack happens. DCSync is invisible without SACLs on the domain partition, and LSASS access is invisible without Sysmon ProcessAccess rules.
- Attackers use credentials from these techniques for lateral movement (Pass-the-Hash, Pass-the-Ticket), which are separate detection challenges with their own log artifacts.